



Module 17: Transactions

Database System Concepts, 7th Ed.

©Silberschatz, Korth and Sudarshan

See www.db-book.com for conditions on re-use



Outline

- Transaction Concept
- Transaction State
- Concurrent Executions
- Serializability
- Recoverability
- Implementation of Isolation
- Transaction Definition in SQL
- Testing for Serializability.



Transaction Concept

- A transaction is a sequence of operations performed as a single logical unit of work. It follows the ACID properties to ensure data consistency and integrity in databases.
- E.g. transaction to transfer \$50 from account A to account B:
 1. **read**(A)
 2. $A := A - 50$
 3. **write**(A)
 4. **read**(B)
 5. $B := B + 50$
 6. **write**(B)
- Two main issues to deal with:
 - Failures of various kinds, such as hardware failures and system crashes
 - Concurrent execution of multiple transactions



Example of Fund Transfer

Example: Suppose you transfer money from Account A to Account B:

Deduct \$100 from Account A.

Add \$100 to Account B.

If any step fails, the transaction is rolled back to prevent data inconsistency. If both succeed, the transaction is committed to the database.

- **Atomicity requirement**

- Ensures a transaction is all-or-nothing.
- If any part of the transaction fails, the entire transaction is rolled back.
- Example: In a bank transfer, if money is deducted from one account, it must also be added to another. If the process fails, no money should be deducted.

- **Durability requirement**

- Ensures that a transaction maintains database integrity by following predefined rules.
- The database must move from one valid state to another without violating constraints (e.g., foreign keys, unique constraints).



Example of Fund Transfer (Cont.)

Example: If a transaction updates a bank balance, the total balance across accounts should remain consistent.

- **Consistency requirement :**
 - Ensures that once a transaction is committed, it is permanently stored.
 - Even if there is a system crash, the committed data must not be lost.
 - Example: After a successful money transfer, the updated balance should persist even after a power failure.



Example of Fund Transfer (Cont.)

- **Isolation requirement** — if between steps 3 and 6, another transaction T2 is allowed to access the partially updated database, it will see an inconsistent database (the sum $A + B$ will be less than it should be).

T1

1. **read**(A)
2. $A := A - 50$
3. **write**(A)
4. **read**(B)
5. $B := B + 50$
6. **write**(B)

T2

read(A), read(B), print(A+B)

- Isolation can be ensured trivially by running transactions **serially**
 - That is, one after the other.
- However, executing multiple transactions concurrently has significant benefits, as we will see later.



ACID Properties

A transaction is a sequence of operations performed as a single logical unit of work. It follows the ACID properties to ensure data consistency and integrity in databases.

- **Atomicity.** A transaction must be fully completed or not executed at all. If any part fails, all changes are undone.
- **Consistency.** A transaction must maintain the correctness of the database before and after execution.
- **Isolation.** Transactions execute independently, ensuring they do not interfere with each other. Each transaction runs independently, and intermediate results are hidden from other transactions until completed.
- **Durability.** Once a transaction is successfully completed, its changes are permanent, even after system failures.

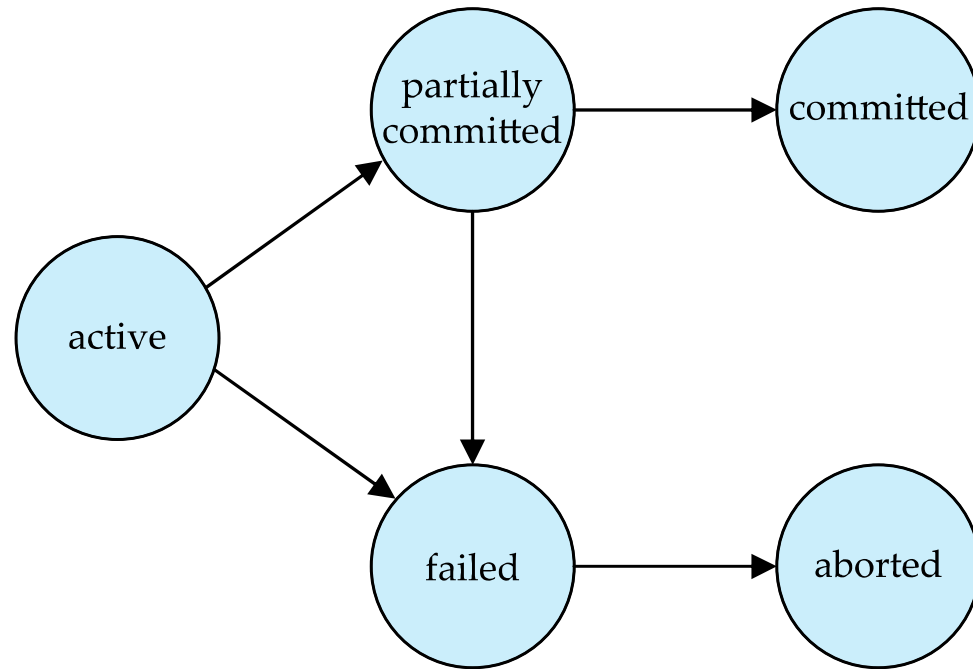


Transaction State

- **Active** – The transaction is currently executing.
- **Partially committed** – The final statement has been executed, but the transaction is not yet complete.
- **Failed** -- The transaction cannot continue due to an error or issue.
- **Aborted** – The transaction is rolled back and the database is returned to its state before the transaction started.
 - After abortion, options are:
 - Restart-- Restart the transaction (if there is no internal error).
 - Kill – End the transaction permanently.
- **Committed** – The transaction has successfully completed, and changes are permanently saved to the database.



Transaction State (Cont.)





Concurrent Executions

Concurrent execution refers to multiple transactions running at the same time in a database system. It improves system performance by allowing multiple users to access and modify data simultaneously,

Advantages of Concurrent Execution

Better Use of CPU and Disk

While one transaction uses the CPU, another can read or write data to the disk. This improves system efficiency and speeds up processing.

Faster Transaction Response

Short transactions don't have to wait for long ones to finish. This reduces the average waiting time for transactions.

Concurrency Control

Special methods are used to prevent errors when multiple transactions run together.

These methods ensure that transactions do not corrupt the database and maintain data consistency.



Schedules

- **Schedule** – A schedule is the order in which instructions from multiple transactions are executed.

Rules of a Schedule:

It must include all instructions from the transactions.

It must maintain the order of instructions inside each transaction.

Types of Transactions in a Schedule:

If a transaction completes successfully, it commits at the end.

If a transaction fails, it aborts and rolls back.



Schedule 1

- Let T_1 transfer \$50 from A to B , and T_2 transfer 10% of the balance from A to B .
- A serial schedule in which T_1 is followed by T_2 :

T_1	T_2
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit



- | T_1 | T_2 |
|--|---|
| read (A)
$A := A - 50$
write (A)
read (B)
$B := B + 50$
write (B)
commit | read (A)
$temp := A * 0.1$
$A := A - temp$
write (A)
read (B)
$B := B + temp$
write (B)
commit |



Schedule 3

- Let T_1 and T_2 be the transactions defined previously. The following schedule is not a serial schedule, but it is *equivalent* to Schedule 1.

T_1	T_2
read (A) $A := A - 50$ write (A)	read (A) $temp := A * 0.1$ $A := A - temp$ write (A)
read (B) $B := B + 50$ write (B) commit	read (B) $B := B + temp$ write (B) commit

In Schedules 1, 2 and 3, the sum $A + B$ is preserved.



- | T_1 | T_2 |
|---|---|
| read (A)
$A := A - 50$ | read (A)
$temp := A * 0.1$
$A := A - temp$
write (A)
read (B) |
| write (A)
read (B)
$B := B + 50$
write (B)
commit | $B := B + temp$
write (B)
commit |



Serializability

Serializability ensures that a concurrent schedule produces the same result as a serial schedule (where transactions execute one after another). This keeps the database consistent.

Key Points:

Assumption → Each transaction maintains database consistency.

Serial Execution → Running transactions one after another keeps the database correct.

Serializable Schedule → A concurrent schedule is serializable if its final result is the same as some serial execution.

Types of Serializability:

conflict serializability → Based on checking for conflicting operations (e.g., read/write conflicts).

view serializability → Based on whether transactions view the same data in the same way as a serial schedule.



Simplified view of transactions

- In this simplified view of transactions:

1. We ignore any operations other than read and write instructions.

No need to worry about computations or processing that happen in between.

2. We assume transactions can do arbitrary work on data in local buffers (temporary storage) between the reads and writes.

3. We focus on the final data that's read and written.

So, the simplified schedules we consider only have read and write instructions.



Conflicting Instructions

- Instructions I_i and I_j of transactions T_i and T_j respectively, **conflict** if and only if there exists some item Q accessed by both I_i and I_j , and at least one of these instructions wrote Q .
 1. $I_i = \text{read}(Q)$, $I_j = \text{read}(Q)$. I_i and I_j don't conflict.
 2. $I_i = \text{read}(Q)$, $I_j = \text{write}(Q)$. They conflict.
 3. $I_i = \text{write}(Q)$, $I_j = \text{read}(Q)$. They conflict
 4. $I_i = \text{write}(Q)$, $I_j = \text{write}(Q)$. They conflict
- Intuitively, a conflict between I_i and I_j forces a (logical) temporal order between them.
 - If I_i and I_j are consecutive in a schedule and they do not conflict, their results would remain the same even if they had been interchanged in the schedule.



Conflict Serializability

- If a schedule S can be transformed into a schedule S' by a series of swaps of non-conflicting instructions, we say that S and S' are **conflict equivalent**.
- We say that a schedule S is **conflict serializable** if it is conflict equivalent to a serial schedule

Conflict Serializability ensures that a schedule (a sequence of transactions) is equivalent to some serial schedule (where transactions execute one after another without interleaving).

Conflicting Operations

Two operations conflict if:

- They belong to different transactions.
- They operate on the same data item.
- At least one of them is a write operation.

Examples of Conflicting Operations

- ✓ Read-Write (RW conflict) $\rightarrow T1: R(A), T2: W(A)$
- ✓ Write-Read (WR conflict) $\rightarrow T1: W(A), T2: R(A)$
- ✓ Write-Write (WW conflict) $\rightarrow T1: W(A), T2: W(A)$
- ✗ Read-Read (RR is NOT a conflict) $\rightarrow T1: R(A), T2: R(A)$



Conflict Serializability (Cont.)

- Schedule 3 can be transformed into Schedule 6, a serial schedule where T_2 follows T_1 , by series of swaps of non-conflicting instructions. Therefore Schedule 3 is **conflict serializable**.

T_1	T_2
read (A) write (A)	
	read (A) write (A)
read (B) write (B)	
	read (B) write (B)

Schedule 3

T_1	T_2
read (A) write (A) read (B) write (B)	
	read (A) write (A) read (B) write (B)

Schedule 6

Example (Recoverable Schedule)

1. T_1 writes A
2. T_2 reads A (from T_1)
3. T_1 commits ✓
4. T_2 commits after T_1 ✓

Since T_2 commits after T_1 commits, it's recoverable.

Example (Not Recoverable Schedule)

1. T_1 writes A
2. T_2 reads A (from T_1)
3. T_2 commits ✗ (Wrong! T_1 hasn't committed yet)
4. T_1 fails (rollback needed!) ✗

Now, T_2 has committed using wrong data, so it's not recoverable.

Checking Conflict Serializability (Precedence Graph / Dependency Graph)

Draw a Directed Graph

Nodes = Transactions (T1, T2, T3, ...)

Edges = Conflicting Dependencies (if one transaction's write affects another transaction's read/write)

Check for Cycles

If the graph has a cycle → NOT conflict serializable

If no cycle → Conflict serializable (i.e., equivalent to some serial execution)

We check **conflicting operations** (same data, at least one is a write):

Conflict Type	Operations	Edge (Direction)
Write-Read (WR)	$T_1: W(A) \rightarrow T_2: R(A)$	$T_1 \rightarrow T_2$
Write-Write (WW)	$T_1: W(A) \rightarrow T_2: W(A)$	$T_1 \rightarrow T_2$
Write-Read (WR)	$T_1: W(B) \rightarrow T_2: R(B)$	$T_1 \rightarrow T_2$
Write-Write (WW)	$T_1: W(B) \rightarrow T_2: W(B)$	$T_1 \rightarrow T_2$

💡 All conflicts show $T_1 \rightarrow T_2$, so we draw a directed edge from T_1 to T_2 .



Conflict Serializability (Cont.)

- Example of a schedule that is not conflict serializable:

T_3	T_4
read (Q)	write (Q)
write (Q)	

- We are unable to swap instructions in the above schedule to obtain either the serial schedule $\langle T_3, T_4 \rangle$, or the serial schedule $\langle T_4, T_3 \rangle$.



View Serializability

- Let S and S' be two schedules with the same set of transactions. S and S' are **view equivalent** if the following three conditions are met, for each data item Q ,
 1. If T_i in S reads the initial value of Q , then T_i must also read the initial value of Q in S' .
 2. If T_i in S reads the value of Q written by T_j , then T_i must read the same value written by T_j in S' .
 3. The transaction that performs the final write on Q in S must also do so in S' .
- As can be seen, view equivalence is also based purely on **reads** and **writes** alone.



View Serializability (Cont.)

- A schedule S is **view serializable** if it is view equivalent to a serial schedule.
- Every conflict serializable schedule is also view serializable.
- Below is a schedule which is view-serializable but *not* conflict serializable.

T_{27}	T_{28}	T_{29}
read (Q)	write (Q)	
write (Q)		write (Q)

- What serial schedule is above equivalent to?
- Every view serializable schedule that is not conflict serializable has **blind writes**.



Other Notions of Serializability

- The schedule below produces same outcome as the serial schedule $\langle T_1, T_5 \rangle$, yet is not conflict equivalent or view equivalent to it.

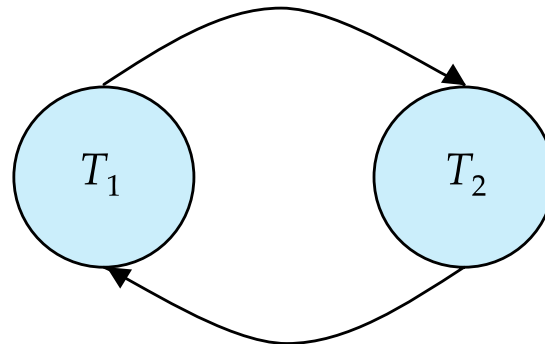
T_1	T_5
read (A) $A := A - 50$ write (A)	
	read (B) $B := B - 10$ write (B)
read (B) $B := B + 50$ write (B)	
	read (A) $A := A + 10$ write (A)

- Determining such equivalence requires analysis of operations other than read and write.



Testing for Serializability

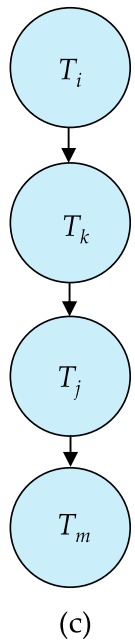
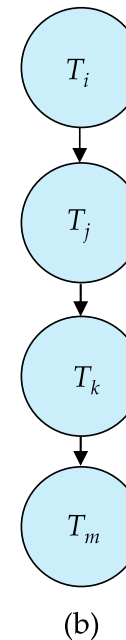
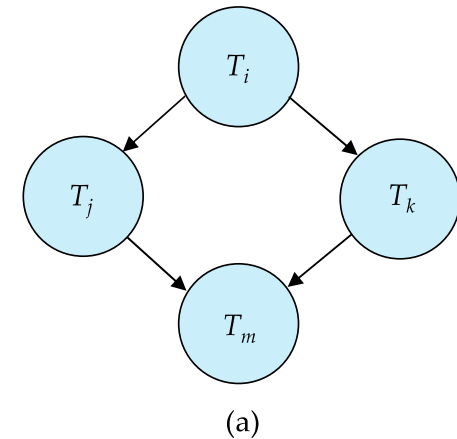
- Consider some schedule of a set of transactions $T_1, T_2, \dots, T_n$
- **Precedence graph** — a direct graph where the vertices are the transactions (names).
- We draw an arc from T_i to T_j if the two transaction conflict, and T_i accessed the data item on which the conflict arose earlier.
- We may label the arc by the item that was accessed.
- **Example 1**





Test for Conflict Serializability

- A schedule is conflict serializable if and only if its precedence graph is acyclic.
- Cycle-detection algorithms exist which take order n^2 time, where n is the number of vertices in the graph.
 - (Better algorithms take order $n + e$ where e is the number of edges.)
- If precedence graph is acyclic, the serializability order can be obtained by a *topological sorting* of the graph.
 - This is a linear order consistent with the partial order of the graph.
 - For example, a serializability order for Schedule A would be
 $T_5 \rightarrow T_1 \rightarrow T_3 \rightarrow T_2 \rightarrow T_4$
 - Are there others?





Test for View Serializability

1. Initial Read Rule

If a transaction T_1 reads a data item X first in the given schedule, it must also be the first to read X in any equivalent serial schedule.

2. Update Rule

If a transaction T_2 reads X after another transaction T_1 has written it, then T_2 must read X from T_1 in the serial schedule too.

3. Final Write Rule

If a transaction T_1 performs the last write on X in the schedule, it must be the last one to write X in the equivalent serial schedule.



Recoverable Schedules

Need to address the effect of transaction failures on concurrently running transactions.

- **Recoverable schedule** A recoverable schedule ensures that if a transaction T_1 reads a value written by another transaction T_2 , then T_2 must commit before T_1 commits to avoid dirty reads.
- The following schedule (Schedule 11) is not recoverable

T_8	T_9
read (A) write (A)	
	read (A) commit
read (B)	

- If T_8 should abort, T_9 would have read (and possibly shown to the user) an inconsistent database state. Hence, database must ensure that schedules are recoverable.

Example (Recoverable Schedule)

1. T_1 writes A
2. T_2 reads A (from T_1)
3. T_1 commits ✓
4. T_2 commits after T_1 ✓

Since T_2 commits after T_1 commits, it's recoverable.

Example (Not Recoverable Schedule)

1. T_1 writes A
2. T_2 reads A (from T_1)
3. T_2 commits ✗ (Wrong! T_1 hasn't committed yet)
4. T_1 fails (rollback needed!) ✗

Now, T_2 has committed using wrong data, so it's not recoverable.



Cascading Rollbacks

- **Cascading rollback** – A cascading rollback occurs when the failure of one transaction forces other dependent transactions to roll back as well. This ensures that the database remains consistent, but it can lead to multiple transactions being undone if they rely on the failed one.

T_{10}	T_{11}	T_{12}
read (A) read (B) write (A)	read (A) write (A)	read (A)
abort		

If T_{10} fails, T_{11} and T_{12} must also be rolled back.

- Can lead to the undoing of a significant amount of work



Cascadeless Schedules

- **Cascadeless schedules** — A cascadeless schedule is one where transactions only read data that has already been committed. This prevents rollbacks from affecting other transactions.
- Every cascadeless schedule is also recoverable
- It is desirable to restrict the schedules to those that are cascadeless



Concurrency Control

- Concurrency control is a technique used in database systems to ensure that multiple transactions can run at the same time without causing errors or inconsistencies.
- A database should make sure that transactions are safe, can recover from failures, and avoid problems like cascading rollbacks. Serial schedules are safe but slow because only one transaction runs at a time. The goal is to create rules that allow transactions to run safely and quickly.



Concurrency Control (Cont.)

- Schedules must be conflict or view serializable, and recoverable, for the sake of database consistency, and preferably cascadeless.
- A policy in which only one transaction can execute at a time generates serial schedules, but provides a poor degree of concurrency.
- Concurrency-control schemes tradeoff between the amount of concurrency they allow and the amount of overhead that they incur.
- Some schemes allow only conflict-serializable schedules to be generated, while others allow view-serializable schedules that are not conflict-serializable.



Concurrency Control vs. Serializability Tests

Concurrency Control: Refers to techniques that manage the execution of multiple transactions at the same time while ensuring data consistency and correctness. It prevents issues like conflicts and ensures that transactions are executed in a way that their results are equivalent to some serial execution. Examples include locking protocols, timestamp ordering, and optimistic concurrency control.

Serializability Tests: These tests check whether a given schedule (a sequence of operations from multiple transactions) is serializable, meaning it produces the same result as if the transactions were executed one at a time in some order. Conflict serializability and view serializability are the two main types of serializability tests.

Key Differences:

Concurrency Control focuses on managing transactions during execution to ensure consistency and avoid conflicts.

Serializability Tests check after execution whether the final result is equivalent to a serial execution.

In short, concurrency control ensures safe and efficient transaction execution, while serializability tests ensure the correctness of the final outcome.



Weak Levels of Consistency

- Some applications are willing to live with weak levels of consistency, allowing schedules that are not serializable
 - E.g., a read-only transaction that wants to get an approximate total balance of all accounts
 - E.g., database statistics computed for query optimization can be approximate (why?)
 - Such transactions need not be serializable with respect to other transactions
- Tradeoff accuracy for performance



Levels of Consistency in SQL-92

- **Serializable** — default
- **Repeatable read** — only committed records to be read.
 - Repeated reads of same record must return same value.
 - However, a transaction may not be serializable – it may find some records inserted by a transaction but not find others.
- **Read committed** — only committed records can be read.
 - Successive reads of record may return different (but committed) values.
- **Read uncommitted** — even uncommitted records may be read.



Levels of Consistency

- Lower degrees of consistency useful for gathering approximate information about the database
- Warning: some database systems do not ensure serializable schedules by default
- E.g., Oracle (and PostgreSQL prior to version 9) by default support a level of consistency called snapshot isolation (not part of the SQL standard)



Transaction Definition in SQL

- In SQL, a transaction begins implicitly.
- A transaction in SQL ends by:
 - **Commit work** commits current transaction and begins a new one.
 - **Rollback work** causes current transaction to abort.
- In almost all database systems, by default, every SQL statement also commits implicitly if it executes successfully
 - Implicit commit can be turned off by a database directive
 - E.g., in JDBC -- `connection.setAutoCommit(false);`
- Isolation level can be set at database level
- Isolation level can be changed at start of transaction
 - E.g. In SQL **set transaction isolation level serializable**
 - E.g. in JDBC -- `connection.setTransactionIsolation(Connection.TRANSACTION_SERIALIZABLE)`



Implementation of Isolation Levels

Overview

- Locking
 - Lock on whole database vs lock on items
 - How long to hold lock?
 - Shared vs exclusive locks
- Timestamps
 - Transaction timestamp assigned e.g. when a transaction begins
 - Data items store two timestamps
 - Read timestamp
 - Write timestamp
 - Timestamps are used to detect out of order accesses
- Multiple versions of each data item
 - Allow transactions to read from a “snapshot” of the database



Transactions as SQL Statements

- E.g. Transaction 1:
select *ID, name* **from** *instructor* **where** *salary* > 90000
- Transaction 2:
insert into *instructor* **values** ('11111', 'James', 'Marketing', 100000)
- Suppose
 - T1 starts, finds tuples *salary* > 90000 using index and locks them
 - And then T2 executes.
 - Do T1 and T2 conflict? Does tuple level locking detect the conflict?
 - Instance of the **phantom phenomenon**
- Also consider T3 below, with Wu's salary = 90000
update *instructor*
set *salary* = *salary* * 1.1
where *name* = 'Wu'
- Key idea: Detect “**predicate**” conflicts, and use some form of “**predicate locking**”



End of Chapter 17

Database System Concepts, 7th Ed.

©Silberschatz, Korth and Sudarshan

See www.db-book.com for conditions on re-use